Kubernetes 싱글 클러스터 구성

(한기원 교육과정)

김영철강사

2024년 12월 14일(토)

Intro : 쿠버네틱스 클러스터 구성을 왜 해야 할까?

1. 쿠버네틱스 강의를 할려면

- 도커 , 쿠버네틱스를 강의할려면 실제 구축을 할 줄 알아야 한다. (현장에서 구축 방식 형태로...)
- 현장에서는 AWS에 대한 관리비용 증가로 인해 Kubernetes 자체 구축하여 사용한다.
- 하이브리드 형태로 구성하여 사용 중

2. AWS EKS , ECS , ECR 이해하기 위해서

- AWS EKS 를 이해하기 위해서는 반드시 선행 교육으로 Kubernetes를 알고 있어야 한다.
- AWS ECS 를 이해하기 위해서는 반드시 선행 교육으로 Kubernetes를 알고 있어야 한다.
- AWS ECR 를 이해하기 위해서는 반드시 선행 교육으로 Kubernetes를 알고 있어야 한다.

3. 자동화된 롤아웃과 롤백 (배포 자동화 롤링업데이트)

- 배포에 대한 기본 개념을 이해하기 위해서
- 쿠버네틱스 기본 네트워크 구성에 대한 개념을 이해하기 위해서 (서비스 , Ingress 등)

4. 부가적으로 쿠버네틱스 자격증 준비를 위해서 (자체적으로 구성하여 시험 대비 명령어 실습)

- CKA
- CKAD

왜 우리는 쿠버네티스 사용할까? “한번더 강조”

1. 자동화된 빈 패킹(bin packing) 컨테이너화된 작업을 실행하는데 사용할 수 있는 쿠버네티스 클러스터 노드를 제공한다. 각 컨테이너가 필요로 하는 CPU와 메모리(RAM)를 쿠버네티스에게 지시한다. 쿠버네티스는 컨테이너를 노드에 맞추어서 리소스를 가장 잘 사용할 수 있도록 해준다.

2. 자동화된 복구(self-healing) 쿠버네티스는 실패한 컨테이너를 다시 시작하고, 컨테이너를 교체하며, '사용자 정의 상태 검사'에 응답하지 않는 컨테이너를 죽이고, 서비스 준비가 끝날 때까지 그러한 과정을 클라이언트에 보여주지 않는다.

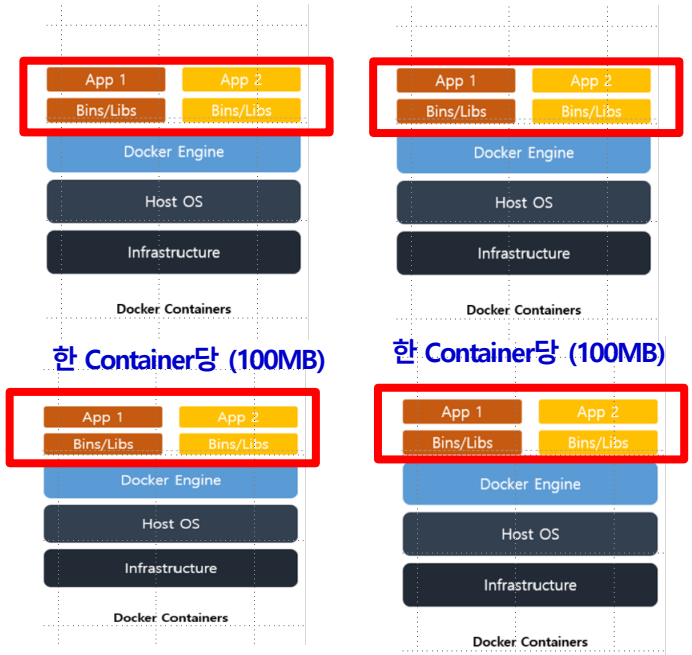
3. 자동화된 롤아웃과 롤백 쿠버네티스를 사용하여 배포된 컨테이너의 원하는 상태를 유지할 수 있으며 현재 상태를 원하는 상태로 설정한 속도에 따라 변경할 수 있다. 예를 들어 쿠버네티스를 자동화해서 배포용 새 컨테이너를 만들고, 기존 컨테이너를 제거하고, 모든 리소스를 새 컨테이너에 적용할 수 있다.

4. 주요 설계 목표는 **컨테이너로 향상된 리소스 활용의 이점을 누리면서도 복잡한 분산 시스템을 쉽게 배포하고 관리할 수 있게 만드는 것**이었다.

5. 쿠버네티스는 **여러 개의** 컨테이너화된 애플리케이션을 **여러 서버(쿠버네티스 클러스터)에** 자동으로 배포, 스케일링 및 관리해주는 오픈소스 시스템

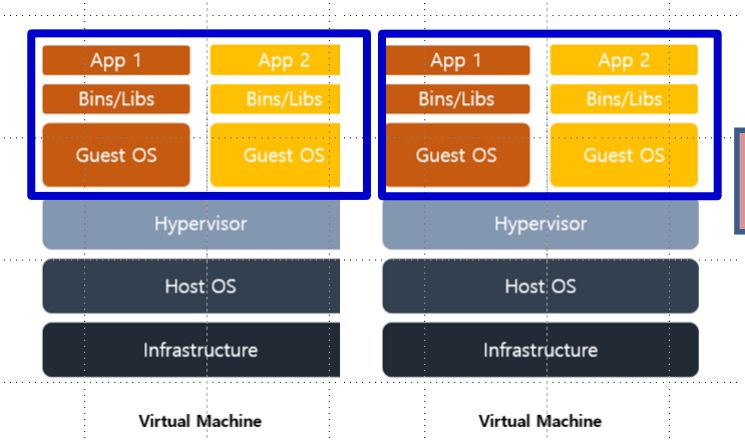
※ 배포의 중요성 :

- 주문 컨테이너의 가격이 1,000원 → 2,000원으로 변경되어 Container 재 배포 해야 한다면 ?

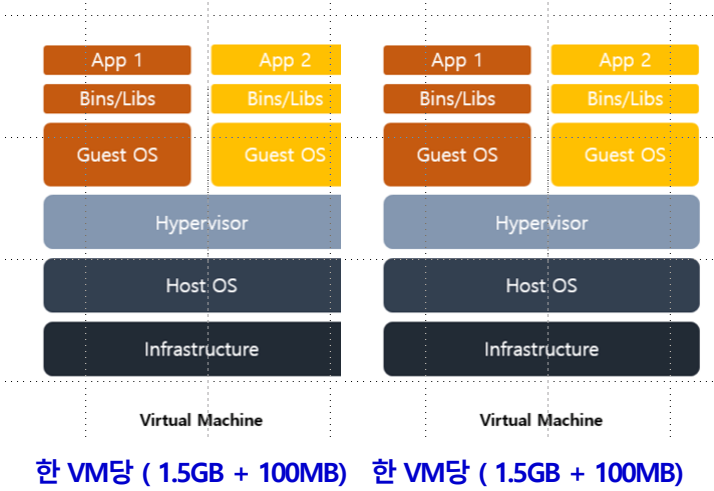


가격 1000원 → 2000원
총 800MB

※ 한 Container당 (100MB) X 8개 = 800 MB

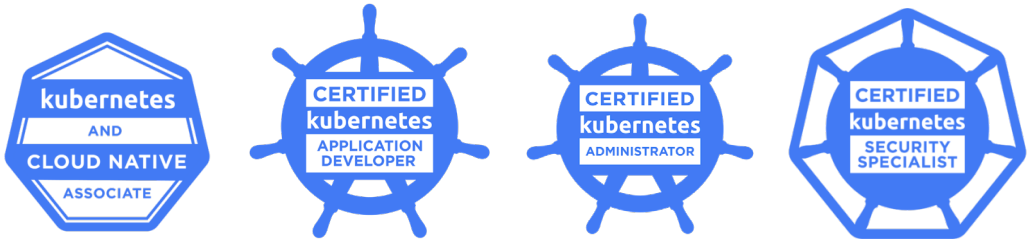


가격 1000원 → 2000원
총 6.4GB



※ 한 VM당 (1.5GB + 100MB) X 4 = 6.4 GB

CNCF 자격증 소개



CKA 자격증 필요

CKA 자격증 소개 (실무 Hands on Lab 시험)

Kubernetes Certified Administrator

Kubernetes **관리자**의 책임을 수행할 수 있는 **기술**, **지식역량**을 추고 있음을 **보증**

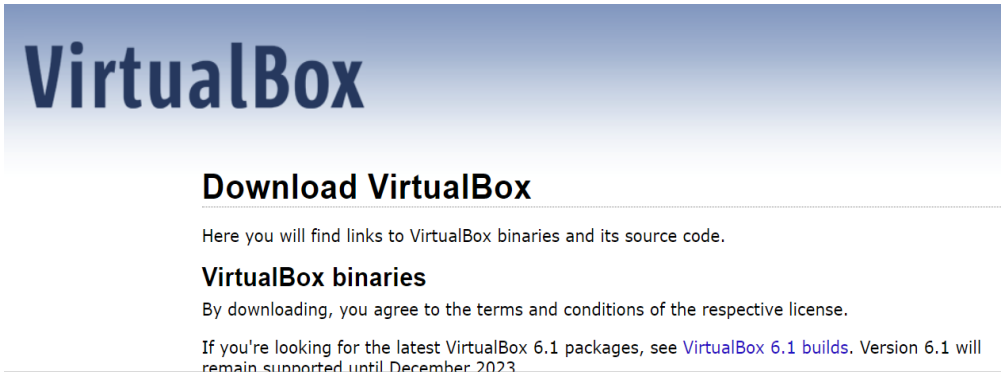
1. VirtualBoX 설치 및 네트워크 구성하기

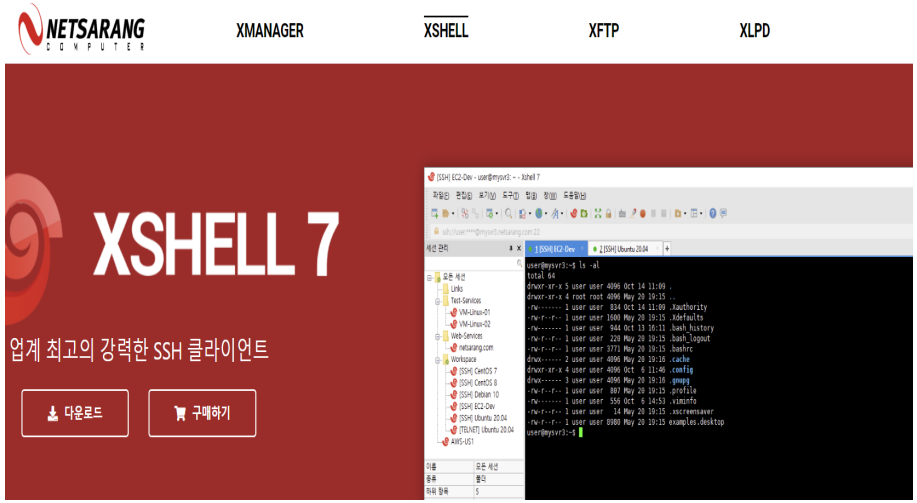
※ 실습 필요한 2가지 소프트웨어 다운로드 필요!!!

1. Windows용 Virtualbox 7.8 버전 다운로드

2. XSHELL 7 버전

(https://www.netsarang.com/ko/xshell/)





※ www.virtualbox.org

virtualbox.org

k8s | 설정 | 동영상강의(CKA) | 교보문고 | k8s_CKA | AWS 관련자료 | 김태민강사 | Docker&Kubernete... | 김영철AW연결계정... | MAC 컴퓨터관리 | 리눅스관련자료

VirtualBox

- About
- Screenshots
- Downloads
- Documentation
  - End-user docs
  - Technical docs
- Contribute
- Community

Welcome to VirtualBox.org!

VirtualBox is a powerful x86 and AMD64/Intel64 virtualization product for enterprise as well as home use. Not only is VirtualBox an extremely feature rich, high performance product for enterprise customers, it is also the only professional solution that is freely available as Open Source Software under the terms of the GNU General Public License (GPL) version 3. See "About VirtualBox" for an introduction.

Presently, VirtualBox runs on Windows, Linux, macOS, and Solaris hosts and supports a large number of guest operating systems including but not limited to Windows (NT 4.0, 2000, XP, Server 2003, Vista, Windows 7, Windows 8, Windows 10), DOS/Windows 3.x, Linux (2.4, 2.6, 3.x and 4.x), Solaris and OpenSolaris, OS/2, and OpenBSD.

VirtualBox is being actively developed with frequent releases and has an ever growing list of features, supported guest operating systems and platforms it runs on. VirtualBox is a community effort backed by a dedicated company: everyone is encouraged to contribute while Oracle ensures the product always meets professional quality criteria.

Download VirtualBox 7.0

News Flash

- **New July 18th, 2023 VirtualBox 7.0.10 released!** Oracle today released a 7.0 maintenance release which improves stability and fixes regressions. See the Changelog for details.
- **New July 18th, 2023 VirtualBox 6.1.46 released!** Oracle today released a 6.1 maintenance release which improves stability and fixes regressions. See the Changelog for details.
- **New April 18th, 2023 VirtualBox 7.0.8 released!** Oracle today released a 7.0 maintenance release which improves stability and fixes regressions. See the Changelog for details.
- **New April 18th, 2023**

Hypervisor 설치하기

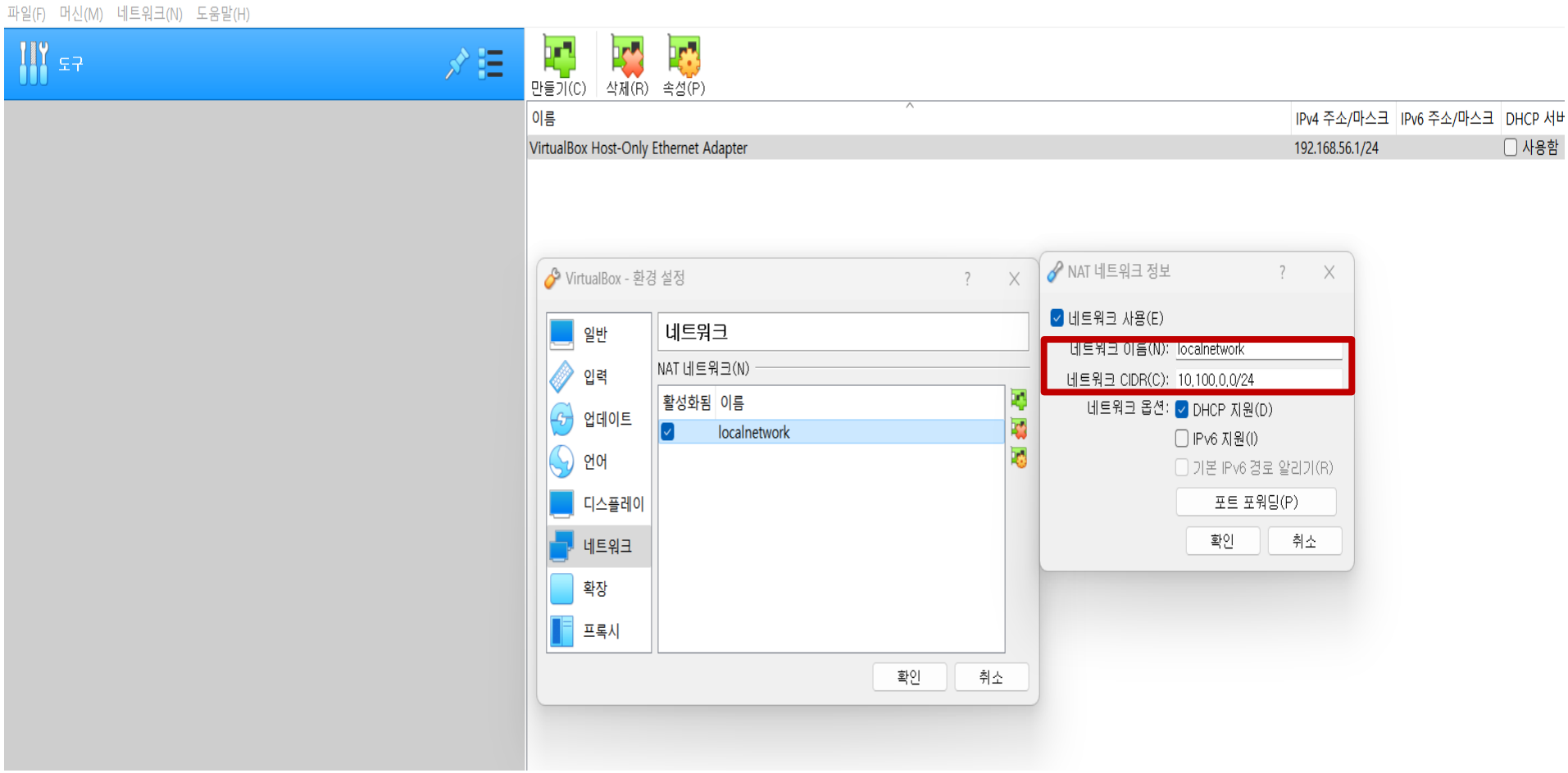
- VirtualBox 다운로드 후 설치
  https://www.virtualbox.org/
- VirtualBox - Network 구성
  - NAT 네트워크 추가 : 파일 - 환경설정 - 네트워크 - 추가
  - 네트워크 이름: localNetwork
  - 네트워크 CIDR: 10.100.0.0/24
  - DHCP 지원
  - 포트포워딩

이름	프로토콜	호스트IP	호스트포트	게스트IP	게스트포트
docker1	TCP	127.0.0.1	105	10.100.0.105	22
docker2	TCP	127.0.0.1	106	10.100.0.106	22

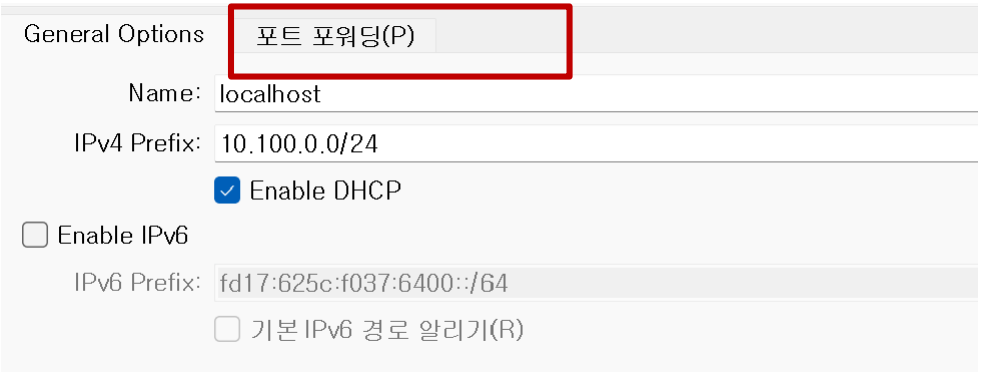
- 가상머신 만들기
  - 이름: docker-ubuntu
    CPU(2core), Memory(2GB), network(localNetwork), disk(20GB)

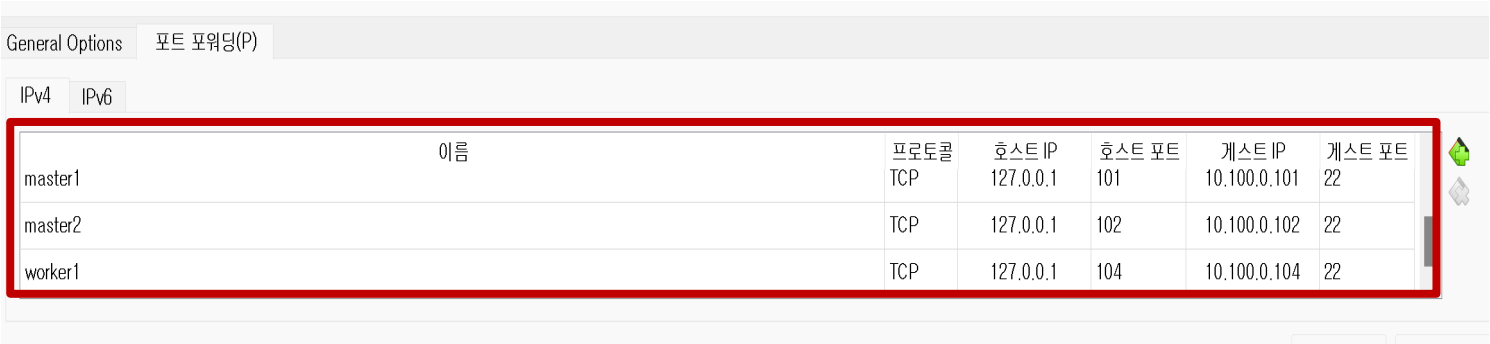
네트워크 구성	IP:10.100.0.105	Netmask:24	GW:10.100.0.1	DNS:10.100.0.1

1. VirtualBox 구성절차서(1)



1. VirtualBox 구성절차서(2)





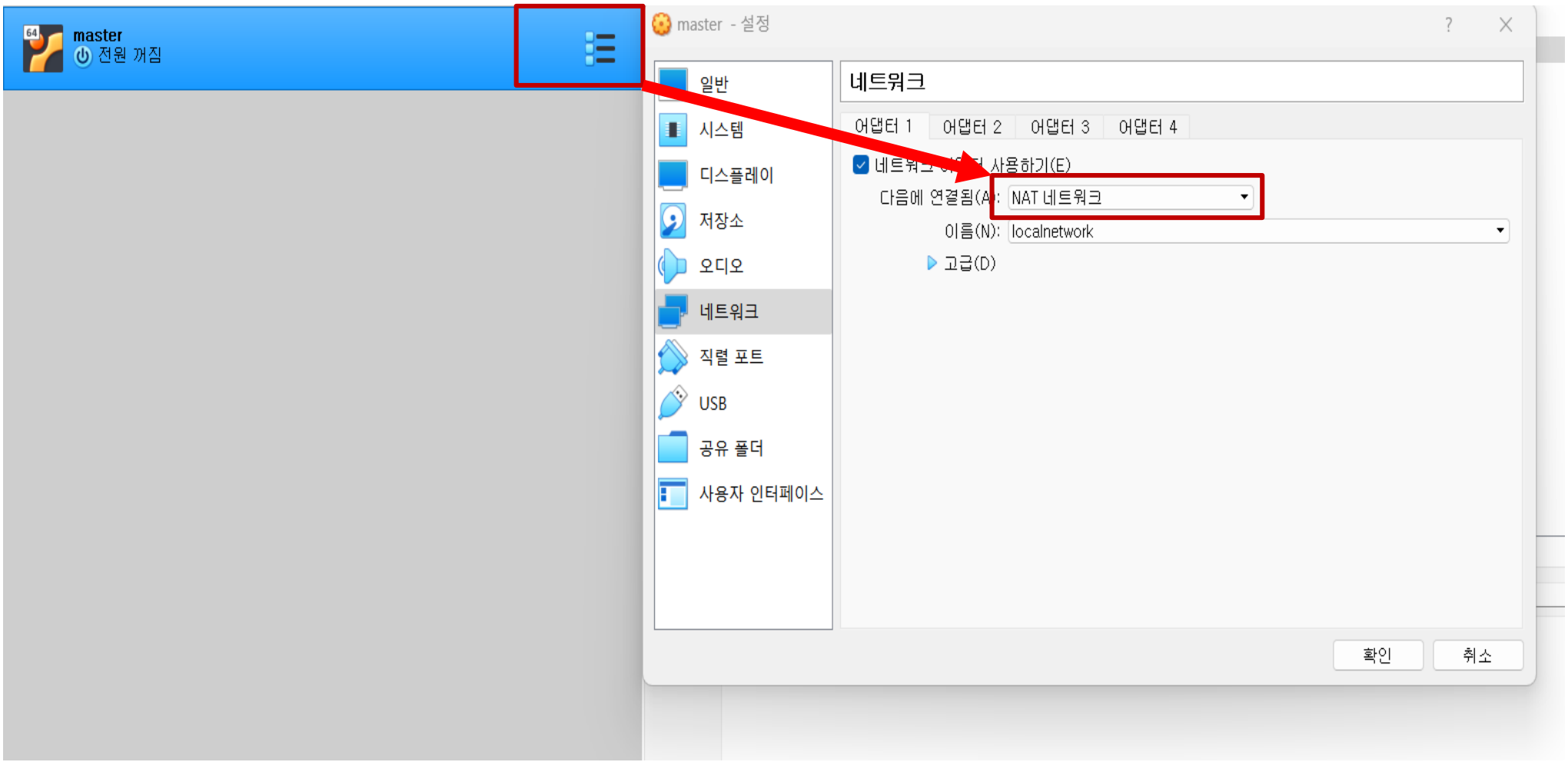
이름	프로토콜	호스트 IP	호스트 포트	게스트 IP	게스트 포트
master1	TCP	127.0.0.1	101	10.100.0.101	22
master2	TCP	127.0.0.1	102	10.100.0.102	22
worker1	TCP	127.0.0.1	104	10.100.0.104	22

K8s-master : 10.100.0.105

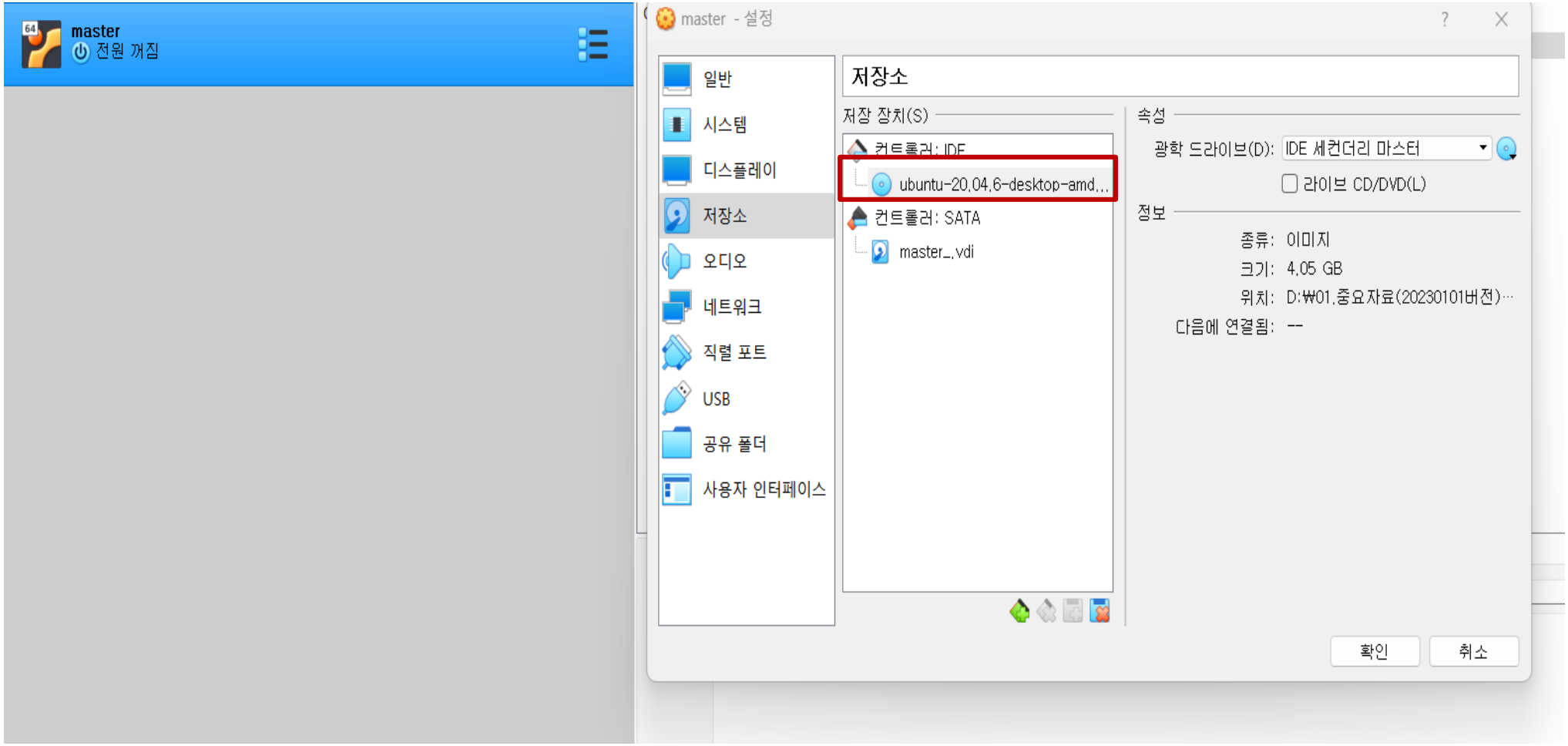
k8s-worker1 : 10.100.0.106

K8s-worker2 : 10.100.0.107

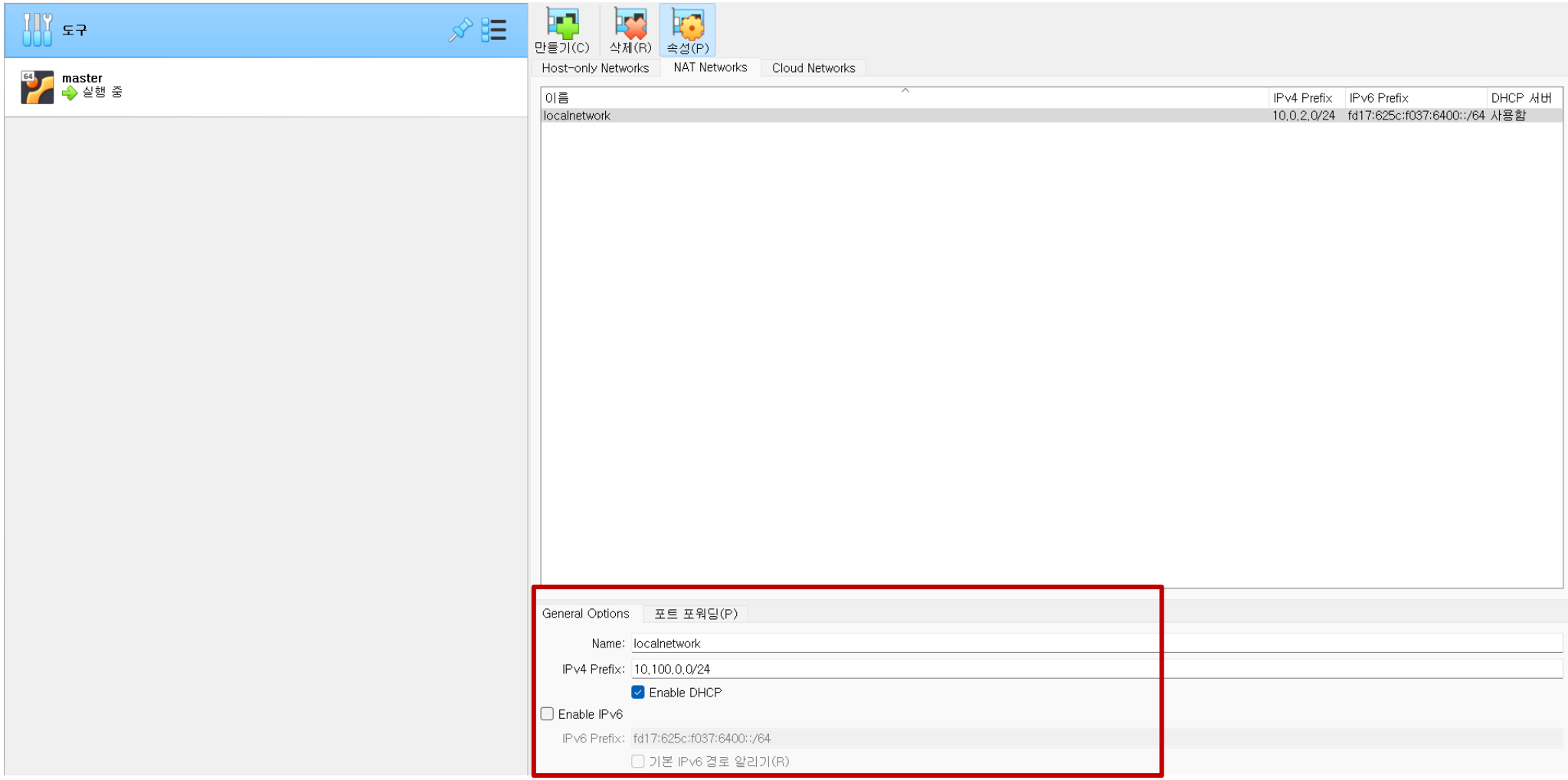
1. VirtualBox 구성절차서(3)



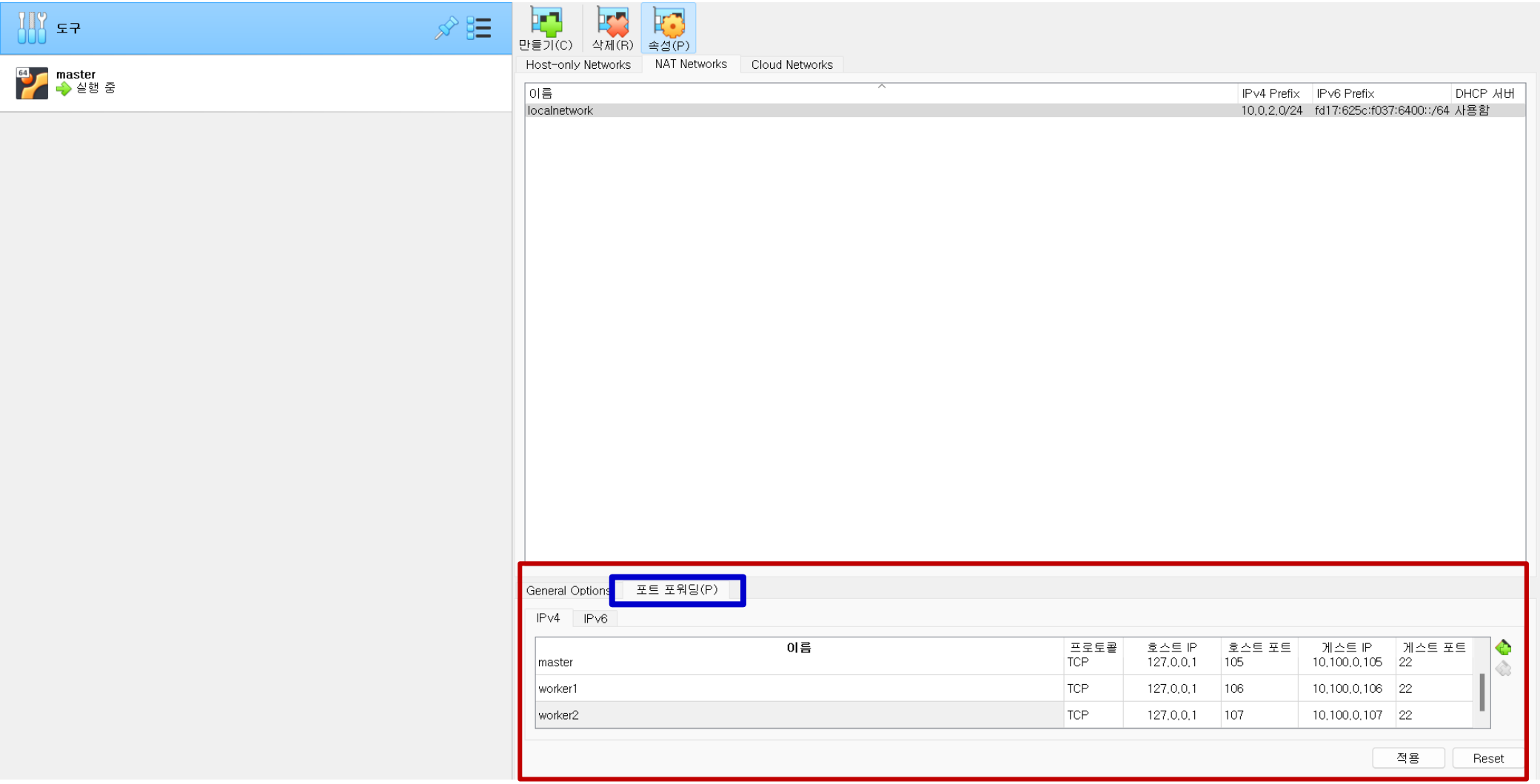
1. VirtualBox 구성절차서(4)



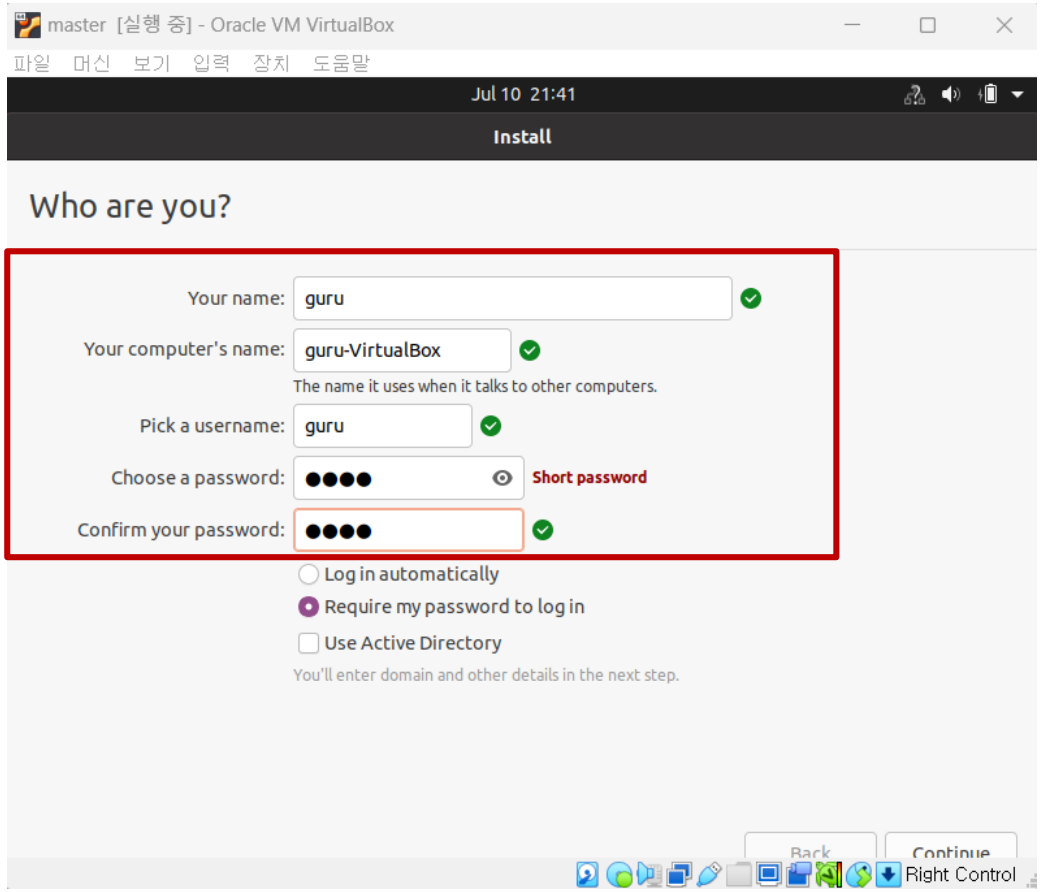
1. VirtualBox 구성절차서(5)



1. VirtualBox 구성절차서(6)

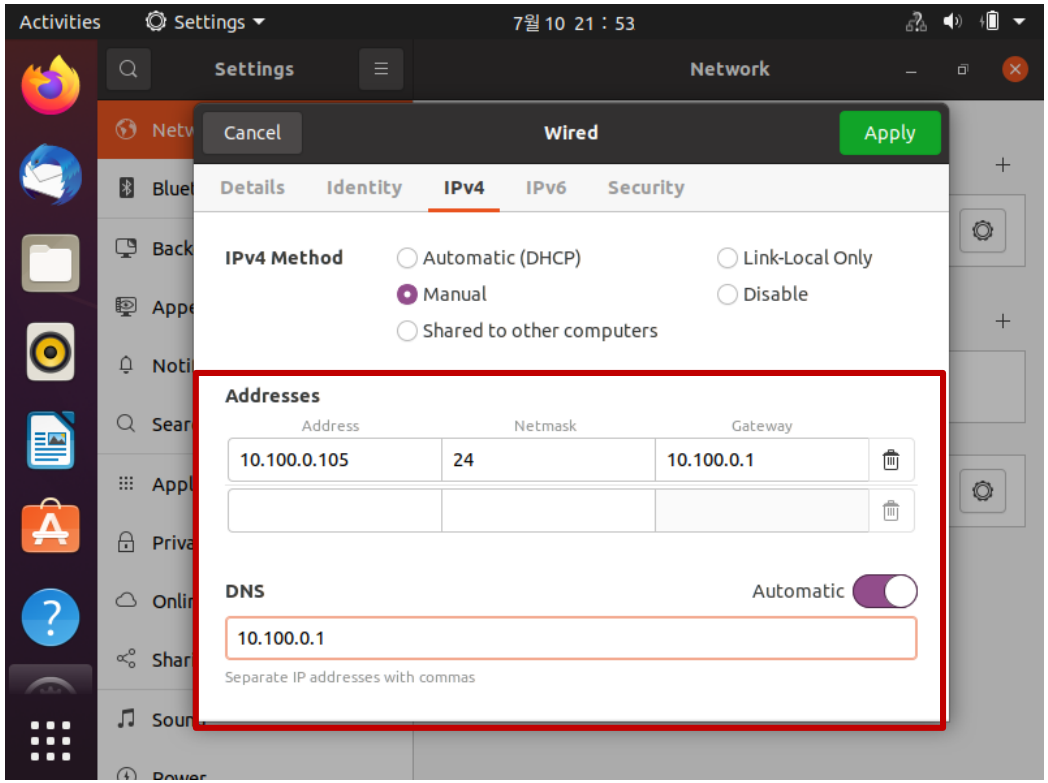


1. VirtualBox 구성절차서(7)



※ 계정정보 guru 암호 : work , root 암호 : password

1. VirtualBox 구성절차서(8)

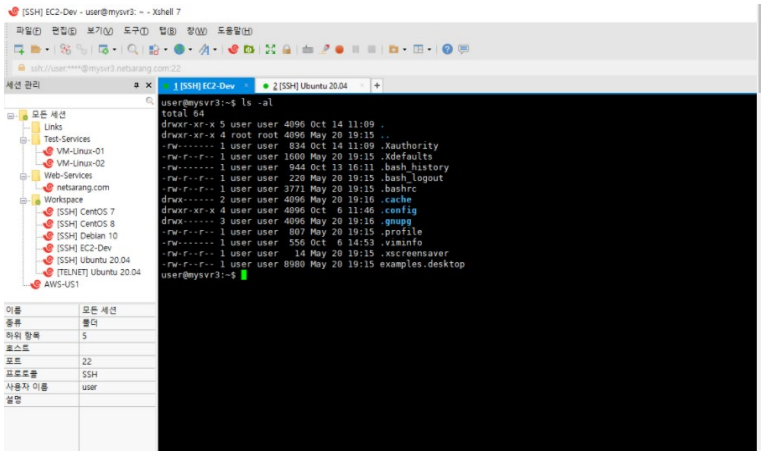


네트워크 구성	IP:10.100.0.105	Netmask:24	GW:10.100.0.1	DNS:10.100.0.1

실습 환경 소개 (설치순서)

1

2



3

ubuntu

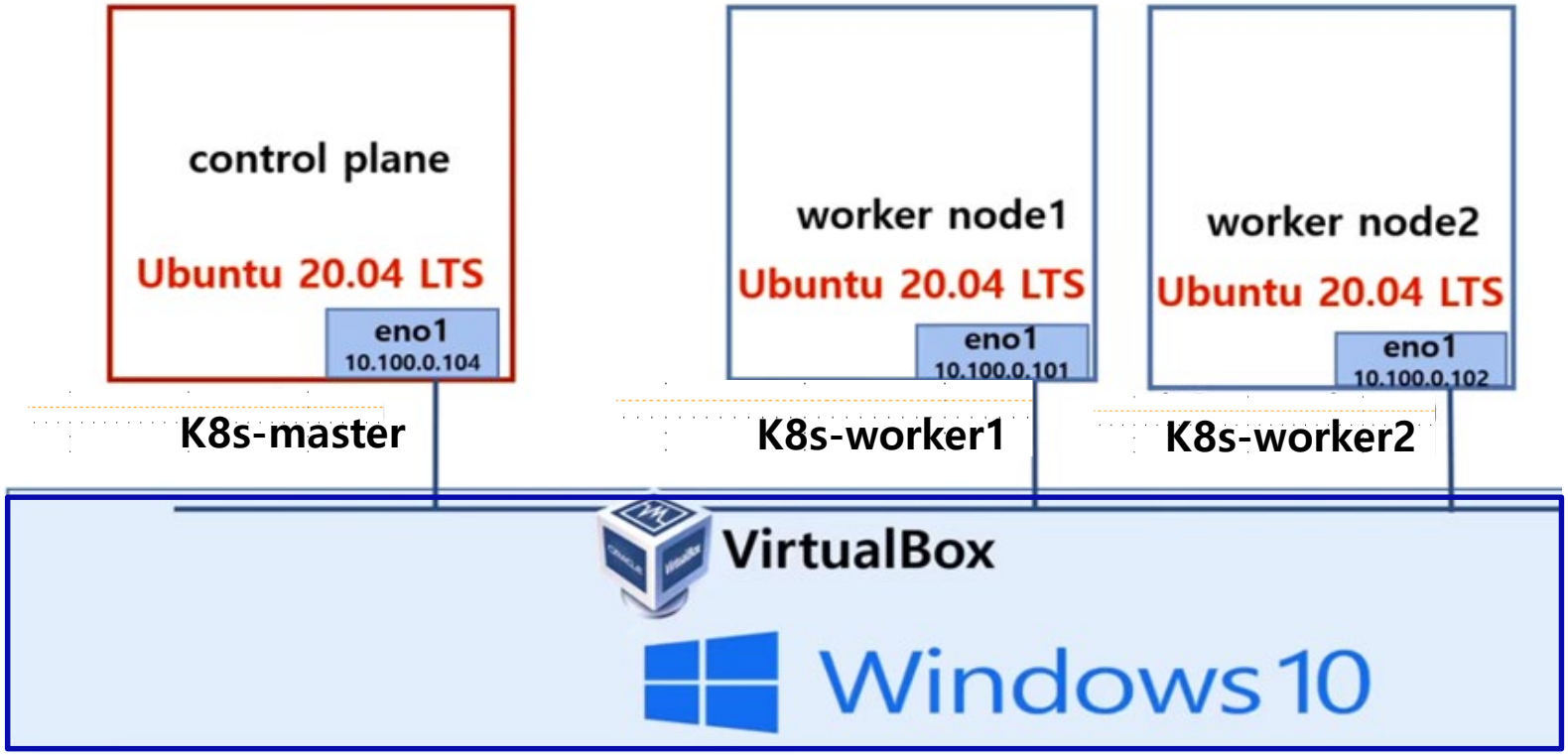
docker

4

kubernetes

Kubernetes 설치 (Contorl Plane : 1대, Data Plane : 2대)

쿠버네티스 클러스터 구성



Kubernetes 싱글 클러스터 구축 가이드 문서

(한기대 교육과정)

김영철강사

2024년 12월14일(토)

#########[kubernetes Cluster 구성 Step by Step 가이드 문서(2024년12월14일)]########

[확인]. 쿠버네티스 클러스터를 구성하기전 요구사항 확인
A compatible Linux host. The Kubernetes project provides generic instructions for Linux distributions based on Debian and Red Hat, and those distributions without a package manager.
2 GB or more of RAM per machine (any less will leave little room for your apps).2 CPUs or more.
Full network connectivity between all machines in the cluster (public or private network is fine)
Unique hostname, MAC address, and product_uuid for every node.
- hostname : # hostname
hostnamectl set-hostname node1
cat /etc/hosts
10.100.0.105 k8s-master
10.100.0.106 k8s-work1
10.100.0.107 k8s-work2

본격적으로 On-premise 싱글 클러스터 환경 구성해 보자

[실행1] 방화벽 해제
sudo systemctl disable –now ufw

[실행2] 스왑 해제
sudo swapoff -a
sudo sed -i '/ swap / s/^₩(.*₩)$/#₩1/g' /etc/fstab

[확인]. Linux 노드의 iptables가 bridged traffic을 정확하게 확인하고 제어 할 수 있도록 br_netfilter 모듈을 load하고 관련된 네트워크 파라미터를 설정한다.

1) #br_netfilter 모듈을 로드하여 관련된 네트워크 파라미터를 설정

```
----------------------------------------------
lsmod | grep br_netfilter
modprobe br_netfilter

lsmod | grep br_netfilter
br_netfilter           24576  0
bridge                155648  1 br_netfilter
----------------------------------------------
```

[실행]3. 모듈과 네트워크 파라미터가 영구적으로 적재되도록 파일에 편집

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
br_netfilter
EOF
----------------------------------------------
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
EOF

sudo sysctl --system
----------------------------------------------
```

[확인]. 네트워크 파라미터 정보 변경 사항 확인

```
----------------------------------------------
# sysctl net.bridge.bridge-nf-call-iptables
net.bridge.bridge-nf-call-iptables = 1
# sysctl net.bridge.bridge-nf-call-ip6tables
net.bridge.bridge-nf-call-ip6tables = 1
```

[확인]. Installing kubeadm, kubelet and kubectl

- **kubeadm: the command to bootstrap the cluster.**
- **kubelet: the component that runs on all of the machines in your cluster and does things like starting pods and containers.**
- **kubectl: the command line util to talk to your cluster.**

Debian-based distributions

[실행]4. Update the apt package index and install packages needed to use the Kubernetes apt repository

```
apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
```

[실행]5. 한줄한줄 명령어로 실행시킨다.

```
mkdir /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.28/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.28/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

[확인]. Update apt package index, install kubelet, kubeadm and kubectl, and pin their version:

[실행]6. Install Kubernetes:

```
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl kubernetes-cni

sudo apt-mark hold kubelet kubeadm kubectl
```

[실행]7. Configuring the kubelet cgroup driver (https://kubernetes.io/docs/setup/production-environment/container-runtimes/)

cgroupfs를 컨테이너 런타임과 kubelet 에 의해서 제어할 수 있도록 구성한다.

```
---------------------------------------------------------------------------------------------------
cat <<EOF | sudo tee /etc/docker/daemon.json
{
  "exec-opts": ["native.cgroupdriver=systemd"],
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "100m"
  },
  "storage-driver": "overlay2"
}
EOF
---------------------------------------------------------------------------------------------------
sudo systemctl enable docker
sudo systemctl daemon-reload
sudo systemctl restart docker
```

[실행]8. 다음 아래 명령어를 순차적으로 입력 (master Node , Worker Node1, Node2 모두에서 실행)

```
sudo rm /etc/containerd/config.toml
sudo systemctl restart containerd
```

[실행]9. (마스터 노드에서만 실행) kubeadm init 명령을 통해서 클러스터를 생성한다.

kubeadm init

```
To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:
export KUBECONFIG=/etc/kubernetes/admin.conf                (root 사용자의 홈디렉토리 쉘 환경파일에 편집)
You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
https://kubernetes.io/docs/concepts/cluster-administration/addons/
Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.137.101:6443 --token k73e80.otf7kbdnbxvc9qvk ₩
        --discovery-token-ca-cert-hash sha256:76507fe082ede68174b6e324f3c72dd0bfd70b78a186c7b46efa26fbf893fd74
```

[실행]10. 마스터 노드에서만 kubeadm init를 실행후 떨어진 결과값 중에서 아래 구문을 실행해 준다. (주의 : 문서 내용이 아닌 본인 화면에 나오는 문장 실행)

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

[실행]11. 쿠버네티스 클러스터에 조인하기 위한 명령어 구문을 저장해둔다.

```
cat > token.txt
kubeadm join 192.168.137.101:6443 --token k73e80.otf7kbdnbxvc9qvk ₩
        --discovery-token-ca-cert-hash sha256:76507fe082ede68174b6e324f3c72dd0bfd70b78a186c7b46efa26fbf893fd74
```

[실행]12. Pod가 서로 통신 할 수 있도록 CNI (Container Network Interface) 기반 Pod 네트워크 추가 기능 구성한다.
네트워크가 설치되기 전에 클러스터 DNS (CoreDNS)가 시작되지 않는다.

```
kubectl apply -f https://github.com/weaveworks/weave/releases/download/v2.8.1/weave-daemonset-k8s.yaml
```
--master 노드에서만 실행해주면 된다.
주의> 강의장에 네트워크 문제로 인해 생성이 안 될수도 있다. (이때 별도의 와이파이로 연결하여 진행을 해본다.)

[확인]. 클러스터 구성 확인

```
root@master:~# kubectl get nodes
NAME     STATUS   ROLES           AGE   VERSION
master   Ready    control-plane   15m   v1.28.3
```

```
root@master:~#  kubectl get pods --all-namespaces
NAMESPACE     NAME                                READY   STATUS    RESTARTS       AGE
kube-system   coredns-5d78c9869d-4bpqx            0/1     Pending   0              15m
kube-system   coredns-5d78c9869d-l5hj8            0/1     Pending   0              15m
kube-system   etcd-master                         1/1     Running   0              16m
kube-system   kube-apiserver-master               1/1     Running   0              16m
kube-system   kube-controller-manager-master      1/1     Running   0              16m
kube-system   kube-proxy-q4gtv                    1/1     Running   0              15m
kube-system   kube-scheduler-master               1/1     Running   0              16m
kube-system   weave-net-wxczl                     2/2     Running   1              (2m2s ago)   2m16s
```

[실행]13. 워커 노드 양쪽에서 kubeadm join 구문을 실행한다

```
 kubeadm join 10.100.0.105:6443 --token p1ixgg.o2nuuihn6cs2zhz1 ₩
                --discovery-token-ca-cert-hash sha256:f8bd392dfc8ec744b673a97e3d16644eb927c399430240601c449e89e76e08cc
```

위 구문을 실행시 아래와 같은 에러 메세지가 발생하면

```
root@work1:~# kubeadm join 10.100.0.105:6443 --token p1ixgg.o2nuuihn6cs2zhz1 ₩
> --discovery-token-ca-cert-hash sha256:f8bd392dfc8ec744b673a97e3d16644eb927c399430240601c449e89e76e08cc
[preflight] Running pre-flight checks
error execution phase preflight: [preflight] Some fatal errors occurred:
                [ERROR CRI]: container runtime is not running: output: time="2023-07-11T00:25:49+09:00" level=fatal msg="validate service connection:
CRI v1 runtime API is not implemented for endpoint ₩"unix:///var/run/containerd/containerd.sock₩": rpc error: code = Unimplemented desc =
unknown service runtime.v1.RuntimeService"
, error: exit status 1
[preflight] If you know what you are doing, you can make a check non-fatal with `--ignore-preflight-errors=...`
To see the stack trace of this error execute with --v=5 or higher
```

[워크노드에서 조인 문제해결 방법]

워크노드에서 아래 명령어를 순차적으로 입력한다.

```
sudo rm /etc/containerd/config.toml
sudo systemctl restart containerd
```

두개의 워크노드에서 다시 조인문장을 아래와 같이 실행한다.

```
kubeadm join 10.100.0.105:6443 --token p1ixgg.o2nuuihn6cs2zhz1 ₩
        --discovery-token-ca-cert-hash sha256:f8bd392dfc8ec744b673a97e3d16644eb927c399430240601c449e89e76e08cc
```

[실행]14. kubectl 자동명령어 실행 스크립트

```
source <(kubectl completion bash)
echo "source <(kubectl completion bash)" >>~/.bashrc

source <(kubeadm completion bash)
echo "source <(kubeadm completion bash)" >>~/.bashrc
```

마스터노드에서 아래 명령어를 실행했을때 아래와 같이 나와야 정상 !!!

```
root@k8s-master:~# kubectl get nodes
NAME          STATUS   ROLES           AGE    VERSION
k8s-master    Ready    control-plane   12m    v1.28.15
k8s-worker1   Ready    <none>          66s    v1.28.15
k8s-worker2   Ready    <none>          55s    v1.28.15
```

※ 에러 발생시 조치사항

==========

마스터노드 조인시 아래와 같은 에러 메세지가 발생한다면

```
root@worker-node1:~# kubeadm join 10.41.2.156:6443 --token 65z741.dd70q5p4d17ch3cv ₩
>         --discovery-token-ca-cert-hash sha256:a6949071dd2204e2ae577f890f5c58b330b0055ee14773f70264789
                [preflight] Running pre-flight checks
                error execution phase preflight: couldn't validate the identity of the API Server: invalid discovery token CA certificate hash: invalid hash
                "sha256:a6949071dd2204e2ae577f890f5c58b330b0055ee14773f70264789", expected a 32 byte SHA-256 hash, found 27 bytes
                To see the stack trace of this error execute with --v=5 or higher
```

==========

아래와 같은 단계진행으로 해결하자.

첫째 : 마스터 노드에서 token list 확인

```
kubeadm token list
```

둘째 : 마스터 노드에서 기존 token 삭제

```
kubeadm token delete va80oy.6rlqvnf6xgqh597o
```

셋째 : 마스터 노드에서 token 재 생성

```
kubeadm token create --print-join-command
```

넷째 : 새로 생성된 token 정보를 가지고 마스터 조인 진행

```
kubeadm join 10.100.0.105:6443 --token utf5dg.09ti6qksojiev5re --discovery-token-ca-cert-hash
sha256:df31e23dc12d189fa6a1951557cd1ef91802640a87a5936b44b33582e0b1f4f1
```

!! 중간에 오류 발생시 참조 !!!

```
$ sudo kubeadm reset
$ sudo rm -rf /var/lib/cni/
$ systemctl daemon-reload
```